

Optimisation in dMod2

Hessian sources, seeding, and the economics of a multi-start

Simon Beyer

2026-09-07

Contents

1	Introduction	2
2	The trust-region core	3
2.1	The subproblem	3
2.2	The step and the radius	4
2.3	Bounds	4
2.4	Termination	4
3	The source of the model Hessian	5
3.1	BFGS and SR1	5
3.2	The handover	6
4	Seed and memory of the approximation	7
5	Multi-start on Boehm	8
5.1	Boehm without the prior	10
6	Bachmann, 113 parameters	11
7	Discussion	12
8	Reproducing the numbers	12
	References	13

1 Introduction

Parameter estimation for ordinary differential equation models is a non-convex problem solved by local optimisation from many starting points. dMod2 [1] provides `trust()`, a trust-region method, and `mstrust()`, which runs it from many starts. This vignette describes their arguments and the behaviour those produce.

The example throughout is the STAT5 dimerisation model of Boehm et al. [2], built as in the shipped example `example_Boehm_JProteomeRes2014.R`: nine parameters on a base-10 logarithmic scale, a weak prior on the six dynamic ones, and bounds of ± 5 . Every table and figure below rests on one shared set of 100 starting points.

```
data(boehm)
mydataL <- as.datalist(boehm)

epo <- "1.25e-7*exp(-Epo_degradation_BaF3*time)"
reactions <- eqnlist() |>
  assignCompartment(nucpApA = "nuc", nucpApB = "nuc", nucpBpB = "nuc",
    volume = "0.45") |>
  addReaction("2*STAT5A", "pApA", paste0("k_phos*", epo, "*STAT5A^2"),
    compartment = "cyt") |>
  addReaction("STAT5A + STAT5B", "pApB", paste0("k_phos*", epo, "*STAT5A*STAT5B"),
    compartment = "cyt") |>
  addReaction("2*STAT5B", "pBpB", paste0("k_phos*", epo, "*STAT5B^2"),
    compartment = "cyt") |>
  addReaction("pApA", "nucpApA", "k_imp_homo*pApA", compartment = "cyt") |>
  addReaction("pApB", "nucpApB", "k_imp_hetero*pApB", compartment = "cyt") |>
  addReaction("pBpB", "nucpBpB", "k_imp_homo*pBpB", compartment = "cyt") |>
  addReaction("nucpApA", "2*STAT5A", "k_exp_homo*nucpApA", compartment = "nuc") |>
  addReaction("nucpApB", "STAT5A + STAT5B", "k_exp_hetero*nucpApB",
    compartment = "nuc") |>
  addReaction("nucpBpB", "2*STAT5B", "k_exp_homo*nucpBpB", compartment = "nuc") |>
  setCompartmentVolume(cyt = "1.4")

# The optimiser can only resolve what the integrator delivers, so the tolerances
# are set explicitly rather than left at their defaults.
myOptionsODE <- list(atol = 1e-8, rtol = 1e-6, maxattempts = 100L, maxsteps = 1e6)
model <- odemodel(reactions, modelname = "boehm_ode", compile = FALSE,
  outdir = .outdir)
x <- Xs(model, optionsOde = myOptionsODE, optionsSens = myOptionsODE)

observables <- eqnvec(
  pSTAT5A_rel = paste0("(100*pApB + 200*pApA*specC17)/",
    "(pApB + STAT5A*specC17 + 2*pApA*specC17)"),
  pSTAT5B_rel = paste0("-(100*pApB - 200*pBpB*(specC17 - 1))/",
    "((STAT5B*(specC17 - 1) - pApB) + 2*pBpB*(specC17 - 1))"),
  rSTAT5A_rel = paste0("(100*pApB + 100*STAT5A*specC17 + 200*pApA*specC17)/",
    "(2*pApB + STAT5A*specC17 + 2*pApA*specC17",
    " - STAT5B*(specC17 - 1) - 2*pBpB*(specC17 - 1))"))
errorModels <- eqnvec(pSTAT5A_rel = "sd_pSTAT5A_rel",
  pSTAT5B_rel = "sd_pSTAT5B_rel",
  rSTAT5A_rel = "sd_rSTAT5A_rel")

g <- Y(observables, x, modelname = "boehm_obs", attach.input = FALSE,
  compile = FALSE, outdir = .outdir)
e <- Y(errorModels, g, modelname = "boehm_err", attach.input = FALSE,
  compile = FALSE, outdir = .outdir)

innerpars <- getParameters(model, g, e)
estimated <- c("Epo_degradation_BaF3", "k_exp_hetero", "k_exp_homo",
  "k_imp_hetero", "k_imp_homo", "k_phos",
```

```

      "sd_pSTAT5A_rel", "sd_pSTAT5B_rel", "sd_rSTAT5A_rel")
p <- eqnvec() |>
  define("x-x", x = innerpars) |>
  define("x-0", x = c("pApA", "pApB", "pBpB", "nucpApA", "nucpApB", "nucpBpB")) |>
  insert("STAT5A ~ 207.6*ratio") |>
  insert("STAT5B ~ 207.6 - 207.6*ratio") |>
  insert("ratio ~ 0.693") |>
  insert("specC17 ~ 0.107") |>
  insert("x-exp10(x)", x = estimated) |>
P(modelname = "boehm_trafo", condition = "Boehm2014", compile = FALSE,
  outdir = .outdir)

compile(g, x, p, e, output = "boehm_vig", cores = 2)

prd      <- g*x*p
outerpars <- getParameters(prd)
pouter   <- structure(rep(-1, length(outerpars)), names = outerpars)
dyn       <- setdiff(outerpars, grep("^sd_", outerpars, value = TRUE))
obj       <- normL2(mydataL, prd, e) + constraintL2(pouter[dyn], sigma = 4)

parlower <- structure(rep(-5, length(outerpars)), names = outerpars)
parupper <- structure(rep( 5, length(outerpars)), names = outerpars)

length(pouter)
#> [1] 9

```

2 The trust-region core

`trust()` minimises a smooth objective by repeatedly solving the constrained quadratic subproblem

$$\min_{\|p\| \leq r} g^\top p + \frac{1}{2} p^\top H p,$$

exactly, through a full eigendecomposition of H [3]. Box constraints are handled by the interior scheme of Coleman and Li [4], in which the distance to the bound that the gradient pushes towards enters the metric of the subproblem. Iterates therefore remain strictly inside the feasible box and never land on its boundary; the `atBound` component of the result reports which parameters a bound is holding.

The subproblem is solved exactly rather than by truncated conjugate gradients because the quasi-Newton updates described below can produce an indefinite H : the Moré-Sorensen solver treats that case directly, whereas a Steihaug-type method can only stop at the trust-region boundary.

2.1 The subproblem

Moré and Sorensen characterise the solution of the subproblem by a scalar. With the eigendecomposition $H = Q\Lambda Q^\top$ and $\gamma = Q^\top g$, a step $p(\lambda) = -Q(\Lambda + \lambda I)^{-1}\gamma$ solves it exactly when $\lambda \geq \max(0, -\lambda_{\min})$ and either $\lambda = 0$ with $\|p\| \leq r$, or $\lambda > 0$ with $\|p\| = r$. The second case is a root of the secular equation

$$\phi(\lambda) = \frac{1}{r} - \frac{1}{\|p(\lambda)\|} = \frac{1}{r} - \left(\sum_i \frac{\gamma_i^2}{(\lambda_i + \lambda)^2} \right)^{-1/2},$$

which is smooth and almost linear on $\lambda > -\lambda_{\min}$, so Newton's method on ϕ converges in a handful of iterations. Writing it this way is what makes the eigendecomposition worth its cost: one decomposition per iteration prices every subsequent λ at $O(n_\theta)$.

The interesting case is the one a positive definite model never reaches. When $\gamma_i = 0$ for every eigenvector of $\lambda_{\min}$, no λ drives $\|p\|$ out to r , and the solution is $p = p(-\lambda_{\min}) + \tau q_{\min}$ with τ chosen so that $\|p\| = r$. This hard case is exactly what an indefinite SR1 model produces, and it is why the exact solver is not a luxury here.

2.2 The step and the radius

One iteration compares the decrease the model promised with the decrease the objective delivered,

$$\rho_k = \frac{f(x_k) - f(x_k + p_k)}{-m_k(p_k)}, \quad m_k(p) = g_k^\top p + \frac{1}{2} p^\top H_k p,$$

and reads the radius off that ratio. A step with $\rho_k < 1/4$ is rejected and the radius contracts to $\min(r_k/4, \|p_k\|/4)$, so a step that was far shorter than the radius shrinks it to its own scale rather than to a quarter of an irrelevant bound. A step with $\rho_k > 3/4$ that used at least nine tenths of the radius doubles it, capped at **rmax**; anything between leaves the radius alone. Only pressing against the boundary earns an increase, since a step that stopped short was limited by the model and not by the region.

Under **stepControl\$nonmonotone** the numerator is measured against a weighted average of past values rather than the current one [5]:

$$Q_{k+1} = \eta Q_k + 1, \quad C_{k+1} = \frac{\eta Q_k C_k + f(x_{k+1})}{Q_{k+1}},$$

with ρ_k scored against C_k . At $\eta = 0$ this is $C_k = f(x_k)$ and the rule above is recovered exactly. A larger η lets the iterate climb out of a narrow valley at the price of accepting steps that raise the objective, so the result reports the best iterate visited rather than the last one. The termination tests keep reading the true objective either way: acceptance is relaxed, convergence is not.

2.3 Bounds

Coleman and Li turn a bound into a change of metric. With v_i the signed distance from x_i to whichever bound the gradient pushes it towards, $D = \text{diag}(|v|^{1/2})$, and J^v the diagonal of $\partial|v|/\partial x$, the subproblem is solved in the scaled variables

$$\hat{g} = Dg, \quad \hat{B} = DHD + \text{diag}(g)J^v,$$

so the model steepens as a bound is approached and the step turns away from it on its own. The first-order optimality measure that **gtol** reads is $\max_i |v_i g_i|$, which vanishes both at an interior stationary point and at a bound the gradient pushes into, and is therefore the right quantity for a box problem. A step that would still leave the box is truncated to a fraction **theta.max** of the distance, reflected off the blocking faces, or replaced by the scaled steepest-descent step, whichever the model rates best.

2.4 Termination

Six tests can end a run, and the **stopReason** component names the one that fired. Which of them apply depends on where the Hessian comes from.

stopReason	criterion	evaluated on
gradient	$\max v \cdot g \leq \mathbf{gtol}$	the current iterate
fvalue	$ f - f_{\text{try}} < \mathbf{ftol}$	the last accepted step
preddiff	predicted decrease $< \mathbf{mtol}$	the last accepted step
step	step norm $< \mathbf{xtol}$	the last accepted step
stagnation	five rejected steps leaving f flat	recent steps

stopReason	criterion	evaluated on
radius	radius < rmin ; reports converged = FALSE	the radius

Only the first tests first-order optimality. Three of the others, **fvalue**, **preddiff** and **step**, read the prediction of the quadratic model, so a Hessian that misrepresents the local curvature can trip them at a point that is not stationary. The handover below exploits that asymmetry; it also accounts for a fit reporting **converged** = TRUE at an implausibly high objective value.

Which tests may end a run follows from the same distinction. While the model is the objective's own $J^T J$, its prediction is a curvature statement and the three apply as given. During a quasi-Newton phase the model is an approximation still being built, so its verdict describes the approximation rather than the iterate; the three are then suppressed and the run ends on **gradient**, on **stagnation**, which counts rejected steps instead of reading the model, or on the radius. The gradient norm is the termination criterion usually recommended for quasi-Newton methods [6].

3 The source of the model Hessian

The argument **hessianMethod** selects where H comes from. The subproblem solver, the scaling and the bound handling are identical in all four cases.

value	Hessian	required from the objective
"gn"	the objective's $J^T J$, rebuilt each iteration	value, gradient, Hessian
"bfgs"	a Powell-damped BFGS update [7]	value, gradient
"sr1"	a symmetric rank-one update, possibly indefinite	value, gradient

hessianFallback names a second source and **fallbackLimit** how often the two may change places. A run is a method and, optionally, a fallback; no particular pair carries a name of its own.

For a sum of squares the Gauss-Newton matrix $J^T J$ is an adequate curvature model wherever the residuals are small, and in dMod2 it is obtained without additional cost: the same Jacobian yields both the gradient and the Hessian, so a gradient and a full derivative evaluation are measurably the same price. This is why "gn" is the default.

The Gauss-Newton matrix omits the term involving residuals times second derivatives. Close to the optimum the quadratic model therefore ceases to improve, and **ftol** or **mtol** fires while the iterate is still short of the minimum. A quasi-Newton approximation does not carry that bias, since it infers curvature from the steps actually taken.

3.1 BFGS and SR1

Both updates read the same secant pair, the step taken and the change in gradient along it,

$$s_k = x_{k+1} - x_k, \quad y_k = \nabla f(x_{k+1}) - \nabla f(x_k),$$

and both ask the new model to reproduce it, $B_{k+1}s_k = y_k$. They differ in what they are willing to give up to do so.

BFGS is the rank-two update that stays positive definite, which it can only do while $s^T y > 0$. Away from a minimum that curvature condition fails, so the pair is damped before use [7]: with

$$\theta = \begin{cases} 1, & s^\top y \geq 0.2 s^\top B s, \\ \frac{0.8 s^\top B s}{s^\top B s - s^\top y}, & \text{otherwise,} \end{cases} \quad \tilde{y} = \theta y + (1 - \theta) B s,$$

the update is

$$B_{k+1} = B_k - \frac{B_k s s^\top B_k}{s^\top B_k s} + \frac{\tilde{y} \tilde{y}^\top}{s^\top \tilde{y}}.$$

Damping interpolates the pair towards Bs , for which the curvature condition holds by construction, so B never loses definiteness. **qnCautious** adds the Li-Fukushima test on top [8]: a pair counts only when $s^\top y > c \|s\| \|y\|$, which discards pairs whose curvature is numerical noise rather than information. **qnSkipped** counts what it rejected.

SR1 is the rank-one update that satisfies the secant equation without insisting on definiteness. With $w = y - B_k s$,

$$B_{k+1} = B_k + \frac{w w^\top}{w^\top s},$$

which is the only symmetric rank-one matrix that does so, and is therefore forced rather than chosen. The price is the denominator: $w^\top s$ can be arbitrarily small, so the update is skipped when $|w^\top s| \leq 10^{-8} \|s\| \|w\|$. SR1 converges to the true Hessian, indefinite regions included, which is useless to a line search and exactly what a trust region wants: the negative curvature it reports is real, and the Moré-Sorensen solver can step along it.

That is also why **qnRejected** is on for SR1. A rejected trial point still supplies a valid (s, y) , since curvature along a step does not depend on whether the step was worth taking, and the trial gradient has been paid for already [6]. BFGS cannot use it: its positive definiteness argument rests on the step having been accepted.

3.2 The handover

A fallback run stays on "gn" until the first of **fvalue**, **preddiff**, **step** or **stagnation** would terminate it, then switches to the fallback source and continues from the last accepted $J^\top J$ as its seed. The **gradient** test remains terminal, since it is a genuine optimality statement rather than a symptom of a stalled model. The construction follows Fides [9], which switches when the trust radius stops updating. The **qnEval** component counts the objective evaluations spent after the switch.

```
set.seed(1L)
start <- as.parvec(msParframe(pouter, n = 1, sd = 3))

single <- .fit("single", list(
  gn      = trust(obj, start, rinit = 0.1, rmax = 10, iterlim = 5000,
    parlower = parlower, parupper = parupper),
  fallback = trust(obj, start, rinit = 0.1, rmax = 10, iterlim = 5000,
    parlower = parlower, parupper = parupper,
    hessianFallback = "bfgs")))

data.frame(method = names(single),
  value = vapply(single, `[`, 0.0, "value"),
  stop = vapply(single, `[`, "", "stopReason"),
  neval = vapply(single, `[`, 0L, "neval"),
  qnEval = vapply(single, `[`, 0L, "qnEval"),
  row.names = NULL)

#>   method   value   stop neval qnEval
#> 1    gn 297.3323 fvalue  105      0
```

```
#> 2 hybrid 297.3260 stagnation 143 38
```

Over the shared starting points of the next section a fallback run spends a few per cent more evaluations than "gn". The `grad` column of the table there counts the starts that end at a stationary point, which a fallback reaches and "gn" does not.

4 Seed and memory of the approximation

Three arguments shape the approximation itself. They are read when the run begins in a quasi-Newton phase, that is for "bfgs" and "sr1"; "gn" ignores them, and a fallback seeds its handover from the Gauss-Newton matrix irrespective of `hessianInit`.

`hessianInit` selects the initial matrix. The default "gn" uses the objective's Hessian at `parinit`. The alternative "identity" starts from the identity and additionally stops `trust()` requesting a Hessian at all, which is what a derivative scheme returning only gradients would be able to supply.

The expectation that a genuine Hessian must be the better seed does not survive a multi-start, which is why the identity is the seed used throughout the rest of this vignette. At a starting point three decades away from the optimum, $J^T J$ is large and severely ill-conditioned. A BFGS update is of rank two and preserves positive definiteness, so it cannot cut such a seed down along the directions where it is wrong by orders of magnitude, and the approximation stays dominated by curvature that was never right; on this model those runs exhaust the iteration cap rather than reaching a termination test. "sr1" is not affected, because its update is of rank one and carries no sign constraint, so it can overwrite bad curvature outright, and the indefinite approximations that follow are what the Moré-Sorensen subproblem solves. `inst/benchmarks` measures both seeds side by side.

`qnMemory` bounds the curvature that is carried forward. The default 0 accumulates every update onto the seed, which is what the rest of this vignette uses. A positive value rebuilds the approximation in each iteration from γI together with the most recent `qnMemory` pairs, where γ is the Shanno-Phua scaling $y^T y / s^T y$ of the newest pair [10]. This is the curvature information of L-BFGS [6], but assembled into the explicit matrix the subproblem requires, so it saves no memory. It is meant for problems where carrying a dense $n_\theta \times n_\theta$ matrix is itself the binding cost, which begins at parameter counts an order of magnitude above anything shown here.

`qnCautious` rejects a BFGS pair whose curvature along the step is negligible, $s^T y \leq \text{qnCautious} \cdot \|s\| \|y\|$ [8]; `qnSkipped` counts the rejections. On this model it changes neither the outcome nor the cost.

```
seeds <- .fit("seeds", lapply(list(
  list(hessianMethod = "bfgs"),
  list(hessianMethod = "bfgs", qnControl = list(hessianInit = "identity")),
  list(hessianMethod = "bfgs",
    qnControl = list(hessianInit = "identity", qnMemory = 10L))),
  function(a) do.call(trust, c(list(obj, start, rinit = 0.1, rmax = 10,
    iterlim = 5000, parlower = parlower,
    parupper = parupper), a))))

data.frame(seed = c("gn", "identity", "identity"),
  memory = c(0L, 0L, 10L),
  value = vapply(seeds, `[`, 0.0, "value"),
  stop = vapply(seeds, `[`, "", "stopReason"),
  neval = vapply(seeds, `[`, 0L, "neval"))

#>      seed memory      value      stop neval
#> 1      gn      0 297.3260 stagnation 107
#> 2 identity      0 279.6607 stagnation 139
#> 3 identity     10 499.4909 stagnation  84
```

5 Multi-start on Boehm

`mstrust()` runs `trust()` from many starting points. Drawing them once with `msParframe()` and passing the resulting frame as `center` ensures that a comparison between methods differs only in the method.

```
set.seed(20260905)
starts <- msParframe(pouter, n = 100, sd = 3)

fitAll <- function(...)
  as.parframe(mstrust(obj, center = starts, fits = nrow(starts),
    cores = .cores, rinit = 0.1, rmax = 10, iterlim = 5000,
    parlower = parlower, parupper = parupper, ...))

source(system.file("benchmarks", "hessianSourceSettings.R", package = "dMod2"))
settings <- hessianSourceSettings[c("gn", "gn_bfgs", "gn_sr1", "bfgs_id", "sr1_id")]

runs <- .fit("multistart", lapply(settings, function(a) do.call(fitAll, a)))
names(runs) <- hessianSourceLabels[names(settings)]
```

A start counts as a hit when it lands within 0.1 of the best value seen anywhere.

```
best <- min(vapply(runs, function(f) min(f$value), 0.0))

summary <- function(f) {
  hits <- sum(f$value <= best + 0.1)
  data.frame(conv      = sum(f$converged),
    grad      = sum(f$stopReason == "gradient"),
    hits      = hits,
    neval      = sum(f$neval),
    medEval    = stats::median(f$neval),
    # The price of one start that reaches the optimum, as reported by
    # the Bachmann run.
    evalPerHit = if (hits > 0) round(sum(f$neval) / hits) else NA_real_)
}

cbind(source = names(runs), do.call(rbind, lapply(runs, summary)),
  row.names = NULL)

#>      source conv grad hits neval medEval evalPerHit
#> 1      gn  100   0    6 35536   219.0     5923
#> 2 gn -> bfgs 100  13    6 38517   244.5     6420
#> 3 gn -> sr1 100  28    6 37290   240.0     6215
#> 4    bfgs_id 100  30   13 11162   120.5      859
#> 5    sr1_id 100  19   34 19726   207.5      580
```

The waterfall plot sorts the final objective values in ascending order. A flat leading plateau indicates repeated arrival at the same optimum, and the height of the first step how well that optimum is separated from its competitors. `plotValues()` draws one collection, shaded by iteration count; the figure below ranks the five separately and colours them by method. A circle marks a converged fit, a triangle one that stopped on any other test.

```
# One panel for several fit collections, each ranked on its own. Drawn back to
# front, so the first one listed is not buried, and without the palette's
# pale yellow.
waterfall <- function(frames, cutoff = Inf) {
  ranked <- do.call(rbind, lapply(rev(names(frames)), function(set) {
    fits <- subset(frames[[set]], value < cutoff)
    fits <- fits[order(fits$value), ]
    data.frame(index = seq_len(nrow(fits)), value = fits$value,
      converged = as.character(fits$converged), set = set)
  )))
}
```



```

ranked$set <- factor(ranked$set, levels = names(frames))
colours <- stats::setNames(dMod_palette(8, "okabe")[-5][seq_along(frames)],
                           names(frames))

ggplot(ranked, aes(index, value, colour = set)) +
  geom_line(aes(group = set), linewidth = 0.3) +
  geom_point(aes(shape = converged), size = 1.3) +
  scale_shape_manual(values = c(`TRUE` = 16, `FALSE` = 17),
                     limits = c("TRUE", "FALSE"), labels = c("yes", "no")) +
  scale_colour_manual(values = colours) +
  labs(x = "index", y = "value", colour = NULL) +
  theme_dMod() +
  theme(legend.position = "bottom", legend.box = "vertical")
}

waterfall(runs, cutoff = 600)

```

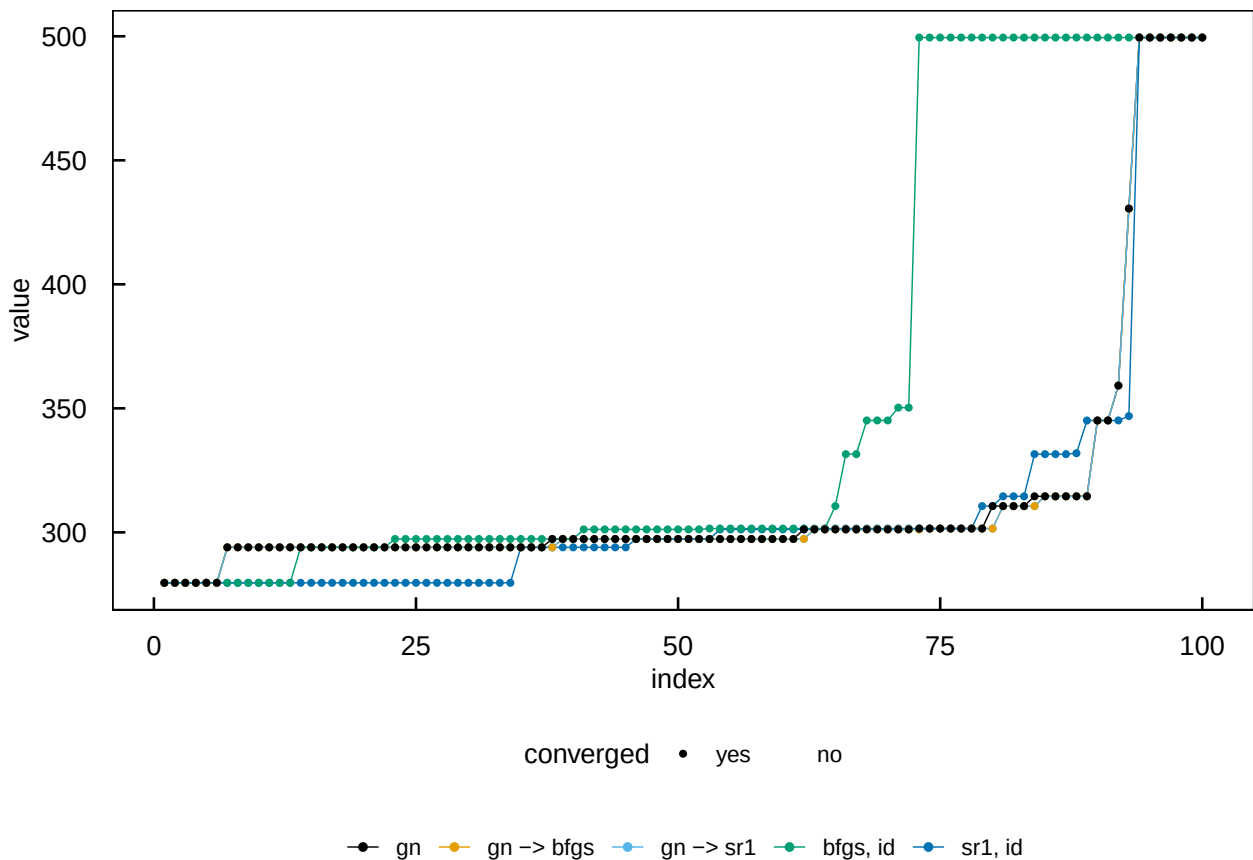


Figure 1: Waterfall plots of the five Hessian sources over the same 100 starting points, each ranked separately. Values above 600 are omitted.

The curves share their starting points and therefore their basins, and they fall into two shapes. Gauss-Newton and the two fallback runs track each other over the whole range, as they must, since a fallback is Gauss-Newton until a soft stop fires; they differ in the leading plateau, which is not flat under Gauss-Newton because the model stops improving before the minimum is reached, whereas the quasi-Newton phase carries those starts onto a common value. The identity-seeded runs have a different shape altogether, with a longer leading plateau: they reach the global optimum from more starting points and abandon more of them further down. Under "bfgs" that trade costs the typical value, under "sr1" it does not.

The `stopReason` of the runs shows the mechanism. Gauss-Newton ends on the model tests, which for it are legitimate. The quasi-Newton runs, which those tests no longer terminate, end on `stagnation` or on `gradient`.

5.1 Boehm without the prior

The prior above keeps a start from running into a bound, and it shapes the comparison. The same 200 starts were therefore also run on the likelihood alone, with the bounds widened to ± 8 . The published optimum is $-2\log L = 276.44$ [11], and a hit ends within 0.1 of it.

source	best	reaching the optimum	median evaluations	evaluations per optimum
"gn"	276.396	28 of 200	98	4236
"gn" to "sr1"	276.396	28 of 200	133	4452
"sr1", identity	276.396	32 of 200	219	4190
"sr1", gn seed	276.396	40 of 200	226	2440
"sr1" to "gn"	276.396	35 of 200	172	2411

```
noprior <- readRDS("boehm_noprior.rds")
waterfall(split(noprior[c("value", "converged")], noprior$source), cutoff = 600)
```

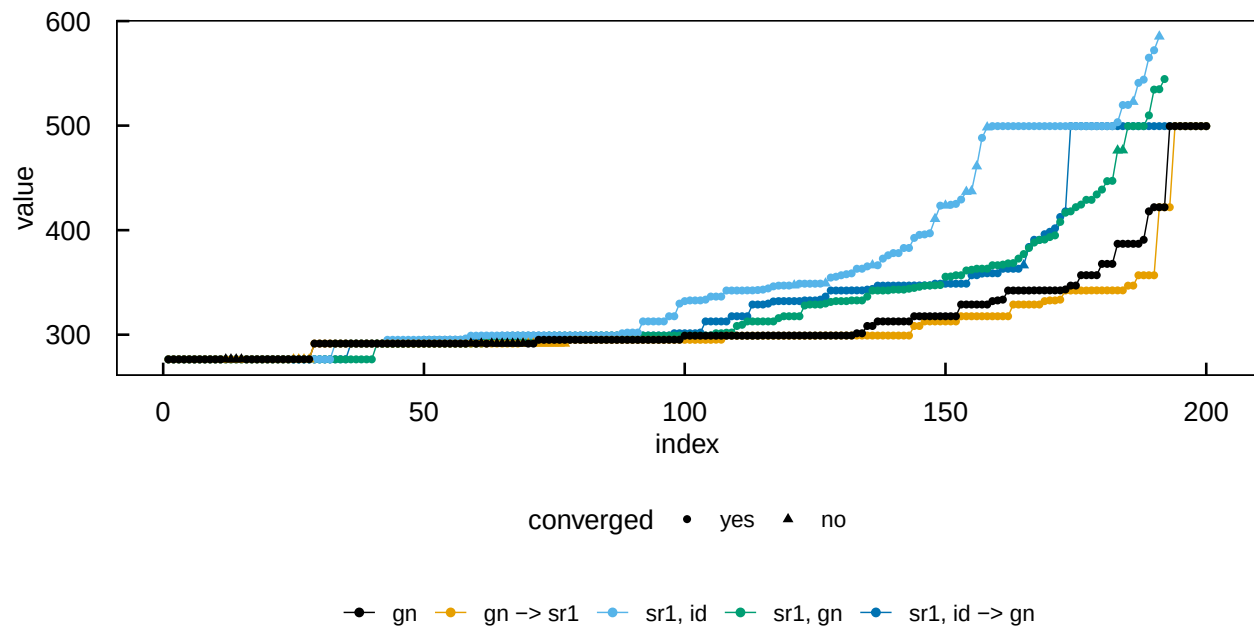


Figure 2: Waterfall plots of five Hessian sources on the Boehm likelihood without a prior, over the same 200 starting points, each ranked separately. Values above 600 are omitted.

Removing the prior costs every source: each still reaches the published optimum, but from fewer starts and at several times the evaluations per one. The ordering changes. "sr1" now reaches the optimum from the most starts and for the fewest evaluations each, and the reversed handover is level with it at a fifth fewer evaluations per start.

The reversed handover also repairs how a run ends. Alone, "sr1" stagnates on 156 of 200 starts; handing back to "gn" turns that into 159 ends on `fvalue` and 11 on `stagnation`, and cuts runs exhausting the iteration cap from 13 to 4. The forward handover does none of this and matches "gn" on every count, as on Bachmann.

6 Bachmann, 113 parameters

A dense quasi-Newton update needs of the order of n_θ accepted steps before it carries curvature along every direction, so its advantage should narrow as the problem grows. Bachmann's JAK2-STAT5 model [12] puts a number on that: 113 estimated parameters, 25 states, 20 observables, 36 conditions. The fits come from a SLURM cluster through `dMod2::distributedComputing()`, one job per source, each an array of 50 tasks running four fits.

The setup needs one deliberate choice. A multi-start over 113 parameters needs a weak prior, or starts run into the bounds instead of converging, but one centre for every parameter is wrong where the published values span twelve decades: at $\sigma = 4$ it costs 27 at the published optimum and moves it. Every parameter further than a decade from the default centre is therefore moved onto the order of magnitude of its published value, which leaves the centre within one decade everywhere and the same $\sigma = 4$ costing 1.04. The objective keeps its published minimum, at -479.26, and a hit ends within 0.1 of it. Starts are drawn around that centre and redrawn where they leave the box.

source	best	reaching the optimum	median evaluations	evaluations per optimum
"gn"	-479.264	83 of 200	141	638
"gn" to "sr1"	-479.264	83 of 200	156	678
"sr1", identity	-479.264	79 of 200	500	1292
"sr1", gn seed	-479.264	65 of 200	592	1926
"sr1" to "gn"	-479.264	79 of 200	490	1275

```
# Cluster fits, so they sit beside the document as data rather than in the
# refit cache: a --refit cannot recompute them.
bachmann <- readRDS("bachmann.rds")
waterfall(split(bachmann[c("value", "converged")], bachmann$source), cutoff = -400)
```

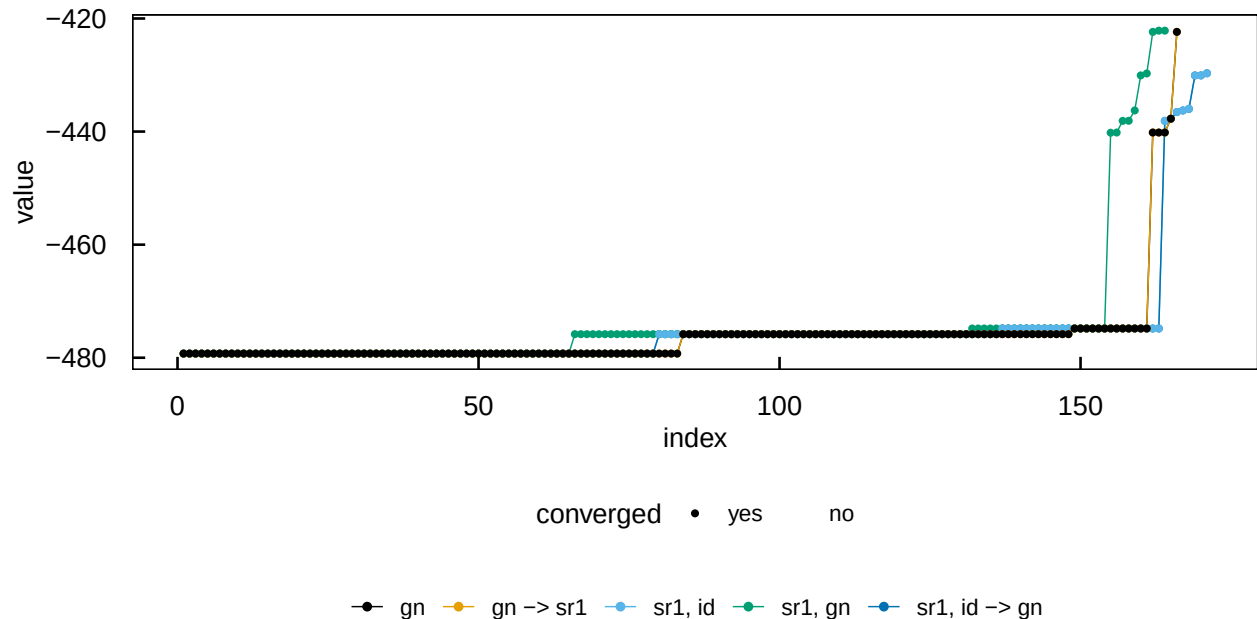


Figure 3: Waterfall plots of five Hessian sources on Bachmann over the same 200 starting points, each ranked separately. Starts ending above -400 are omitted.

Every source reaches the same best value, and the counts at 0.1 and at 1.0 from the optimum are identical, so a start either finds the published minimum or ends far from it. No source finds an optimum another

misses. What separates them is price: Gauss-Newton reaches the optimum for a third of what the cheapest quasi-Newton run pays, reversing the ordering seen at nine parameters.

Two differences are structural rather than statistical.

An identity seed beats a Gauss-Newton seed for "sr1", 79 starts against 65. Seeding an approximation that is about to be rebuilt from scratch with a matrix from a point it is about to leave carries the wrong curvature into the first steps, and SR1 has no damping to discard it.

The handovers do what their construction says and no more. Forward, the model tests never fire again and 197 of 200 runs end on stagnation, at the values "gn" reaches alone. Reversed, the run ends on `fvalue` 140 times instead, because the Gauss-Newton model is a legitimate stopping test again. Neither changes which optimum is found: matched start by start, "gn" and the forward handover differ by 3×10^{-6} . By the time a soft stop fires the basin is chosen, and the handover only decides how the run ends inside it.

7 Discussion

The five sources differ in what a single start buys. "gn" and the two fallback runs reach the published optimum from the same number of starts; a fallback changes only how a run ends, moving stops off the model tests and onto the gradient, and an "sr1" fallback moves more of them than a "bfgs" one.

The source that separates is "sr1" seeded from the identity. It reaches the published optimum from several times as many starts as "gn" and at a fraction of the evaluations each one costs.

`parscale`, the diagonal preconditioner, is off by default and stays off here, and the quasi-Newton seed is not scaled either. Neither has anything to correct when parameters are already on a logarithmic scale, which is the normal case in dMod2, and the ill-conditioning of sloppy models lies along rotated directions [13] that no diagonal scaling reaches. Scaling the seed is worse than idle: it reads one curvature from the first accepted pair of a run and applies it to every direction, which a spectrum spread over decades does not support.

Three settings are not a set of models, but they are enough to separate what travels from what does not.

The ordering of the sources does not travel. Gauss-Newton reaches the optimum for a third of what the cheapest quasi-Newton run pays on Bachmann; on Boehm "sr1" is cheapest, by an order of magnitude with the prior and by a factor of two without it. The direction is the one the theory gives: where the residuals at the optimum are small, $J^T J$ is close to the true Hessian and a source that infers curvature from steps pays for information it does not need. Where they are large the omitted term is not, the Gauss-Newton model stops improving early, and its own tests fire far from the minimum.

Three things do travel. No source reaches an optimum another misses; every comparison here separates on price, not on which minimum is found. A handover does not change which basin a run ends in, because by the time a soft stop fires the basin has been chosen: matched start by start on Bachmann, "gn" and the forward handover differ by 3×10^{-6} . And a quasi-Newton phase costs of the order of n_θ accepted steps before it carries curvature everywhere, which is why its price per optimum grows with the parameter count.

The two handovers therefore buy different things, and neither buys an optimum. The forward one changes only where inside a basin a run stops. The reversed one restores a legitimate stopping test: on Boehm without a prior it turns 156 stagnations into 159 ends on `fvalue` and cuts runs hitting the iteration cap from 13 to 4.

8 Reproducing the numbers

The nine-parameter fits behind the tables and figures are stored in `dev/optimisation/results.rds` and reused, so this document renders in seconds and reproduces exactly what it shows; `dev/render-optimisation.R --refit` recomputes them.

The two heavy comparisons are too expensive for a render and sit beside the document as data instead, one file each, with the script that produced them:

- `boehm_noprior.rds` comes from `bench_hessianSource.R` under `inst/benchmarks` with the environment variable `BOEHM_NOPRIOR` set, and takes minutes on one machine.
- `bachmann.rds` comes from `bachmann_hessianSource.R` under `dev/optimisation`, first with `BACHMANN_SUBMIT` and then with `BACHMANN_COLLECT`. It needs a SLURM cluster: five job arrays of 50 tasks, four fits each.

`bench_hessianSource.R` without that variable is the nine-parameter comparison of the earlier sections, over the whole catalogue rather than the subset shown here and with the per-start `stopReason` tables.

References

- [1] D. Kaschek, W. Mader, M. Fehling-Kaschek, M. Rosenblatt, and J. Timmer, “dMod: Efficient parameter estimation and uncertainty analysis for dynamic models,” *Journal of Statistical Software*, vol. 88, no. 10, pp. 1–32, 2019, doi: 10.18637/jss.v088.i10.
- [2] M. E. Boehm, L. Adlung, M. Schilling, S. Roth, U. Klingmüller, and W. D. Lehmann, “Identification of isoform-specific dynamics in phosphorylation-dependent STAT5 dimerization by quantitative mass spectrometry and mathematical modeling,” *Journal of Proteome Research*, vol. 13, no. 12, pp. 5685–5694, 2014, doi: 10.1021/pr5006923.
- [3] J. J. Moré and D. C. Sorensen, “Computing a trust region step,” *SIAM Journal on Scientific and Statistical Computing*, vol. 4, no. 3, pp. 553–572, 1983, doi: 10.1137/0904038.
- [4] T. F. Coleman and Y. Li, “An interior trust region approach for nonlinear minimization subject to bounds,” *SIAM Journal on Optimization*, vol. 6, no. 2, pp. 418–445, 1996, doi: 10.1137/0806023.
- [5] H. Zhang and W. W. Hager, “A nonmonotone line search technique and its application to unconstrained optimization,” *SIAM Journal on Optimization*, vol. 14, no. 4, pp. 1043–1056, 2004, doi: 10.1137/S1052623403428208.
- [6] J. Nocedal and S. J. Wright, *Numerical optimization*, 2nd ed. Springer, 2006. doi: 10.1007/978-0-387-40065-5.
- [7] M. J. D. Powell, “A fast algorithm for nonlinearly constrained optimization calculations,” *Lecture Notes in Mathematics*, vol. 630, pp. 144–157, 1978, doi: 10.1007/BFb0067703.
- [8] D.-H. Li and M. Fukushima, “A modified BFGS method and its global convergence in nonconvex minimization,” *Journal of Computational and Applied Mathematics*, vol. 129, no. 1–2, pp. 15–35, 2001, doi: 10.1016/S0377-0427(00)00540-9.
- [9] F. Fröhlich and P. K. Sorger, “Fides: Reliable trust-region optimization for parameter estimation of ODE models,” *PLOS Computational Biology*, vol. 18, no. 7, p. e1010322, 2022, doi: 10.1371/journal.pcbi.1010322.
- [10] D. F. Shanno and K.-H. Phua, “Matrix conditioning and nonlinear optimization,” *Mathematical Programming*, vol. 14, no. 1, pp. 149–160, 1978, doi: 10.1007/BF01588962.
- [11] H. Hass, C. Loos, E. Raimúndez-Álvarez, J. Timmer, J. Hasenauer, and C. Kreutz, “Benchmark problems for dynamic modeling of intracellular processes,” *Bioinformatics*, vol. 35, no. 17, pp. 3073–3082, 2019, doi: 10.1093/bioinformatics/btz020.
- [12] J. Bachmann *et al.*, “Division of labor by dual feedback regulators controls JAK2/STAT5 signaling over broad ligand range,” *Molecular Systems Biology*, vol. 7, no. 1, p. 516, 2011, doi: 10.1038/msb.2011.50.
- [13] R. N. Gutenkunst, J. J. Waterfall, F. P. Casey, K. S. Brown, C. R. Myers, and J. P. Sethna, “Universally sloppy parameter sensitivities in systems biology models,” *PLOS Computational Biology*, vol. 3, no. 10, p. e189, 2007, doi: 10.1371/journal.pcbi.0030189.